Data Structures

Linked Lists

Kaynat Rana

Abstract Data Type

- An ADT is a collection of data and a set of operations on that data.

- Specifications indicate what ADT operations do, but not how to implement them

Abstract Data Type

- For example, you wanted a data structure to store both the names and salaries of a group of employees. You could use the following C++ statements:

```
const int MAX_NUMBER = 500;
string names[MAX_NUMBER];
Double salaries[MAX_NUMBER];
```

- Here the employee *names[i],* has a salary of *salaries[i]*. The two arrays *names* and *salaries* together form a data structure, yet C++ has no single data type to describe it.

Abstract Data Type

- When a program must perform data operations that are not directly supported by the language, you should first design an abstract data type and carefully specify what the ADT operations are to do.

- Then —*and only then*—should you implement the operations with a data structure.

ADT vs Data Structures

- An abstract data type is a collection of data and a set of operations on that data.

- A data structure is a construct within a programming language that stores a collection of data.

ADT Implementation

- A program should not depend on the details of an ADT's implementation

- That is, you request the ADT operations to manipulate the data in the data structure, and they pass the results of these manipulations back to you.

- Using an ADT is like using a vending machine.

ADT List Operations

- Create an empty list.
- Destroy a list.
- Determine whether a list is empty.
- Determine the number of items on a list.
- Insert an item at a given position in the list.
- Delete the item at a given position in the list.
- Look at (retrieve) the item at a given position in the list.

ADT List Operations

- To begin, consider how you can construct this list by using the ADT list operations.
- One way is first to create an empty list *aList* and then use a series of insertion operations to append successively the items to the list as follows:

    - *aList.createList()*
    - *aList.insert(1, milk, success)*
    - *aList.insert(2, eggs, success)*
    - *aList.insert(3, butter, success)*
    - *aList.insert(4, apples, success)*

    Milk,eggs,butter,apples.

ADT List Operations

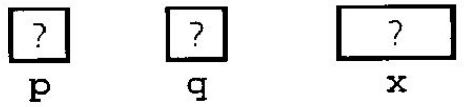
- *aList.remove(4, success)*

- *Now list becomes*

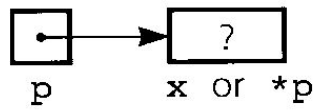
  Milk,eggs,butter

Pointer Operations (Reminder)

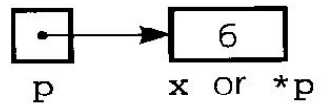
```
(a) int *p, *q;
    int      x;
```



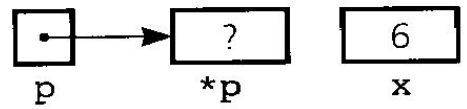

```
(b) p = &x;
```



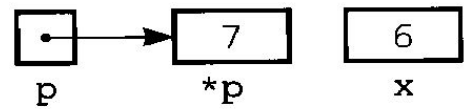

```
(c) *p = 6;
```



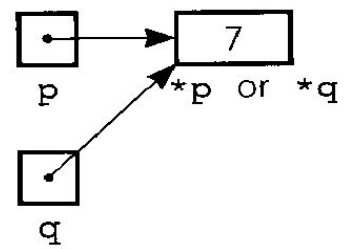

```
(d) p = new int;
```




```
(e) *p = 7;
```




```
(f) q = p;
```

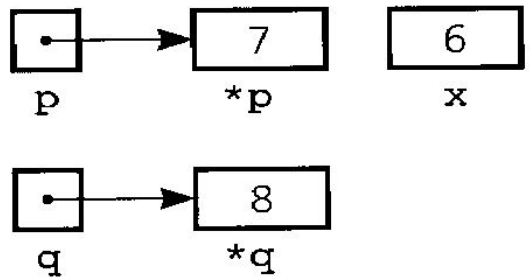


(a) Declaring pointer variables; (b) pointing to statically allocated memory; (c) assigning a value; (d) allocating memory dynamically;

(e) assigning a value;
(f) copying a pointer;

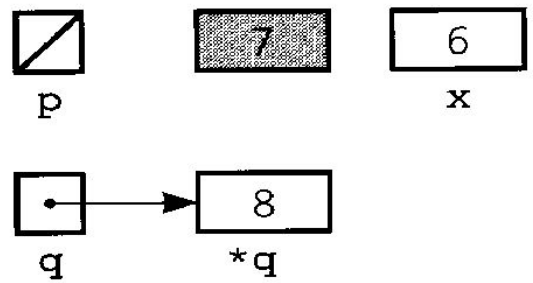
Pointer Operations (Reminder)

(g) `q = new int;`
`*q = 8;`



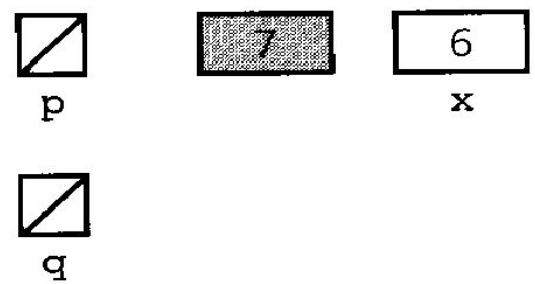
(g) allocating memory dynamically and assigning a value;

(h) `p = NULL;`



(h) assigning ***NULL*** to a pointer variable;

(i) **`delete q;`**
`q = NULL;`



(i) deallocating memory

Pointer Operations (Reminder)

- To avoid memory leak
  - Delete q (Delete returns the memory to the system for reuse)

- Delete q does not deallocate q, it leaves q undefined
  - q=NULL (The cell to which q pointed previously is now lost. You can not reference it)

ADT Linked List

- Array can not be used to implement list in strict terms in most programming languages because of its fixed size. List should have a dynamic size.

- If you need to insert a new item, you simply find its place in the list and set two pointers.

- If you want to delete an Item. Find the item and change a pointer to bypass the item.

ADT Linked List

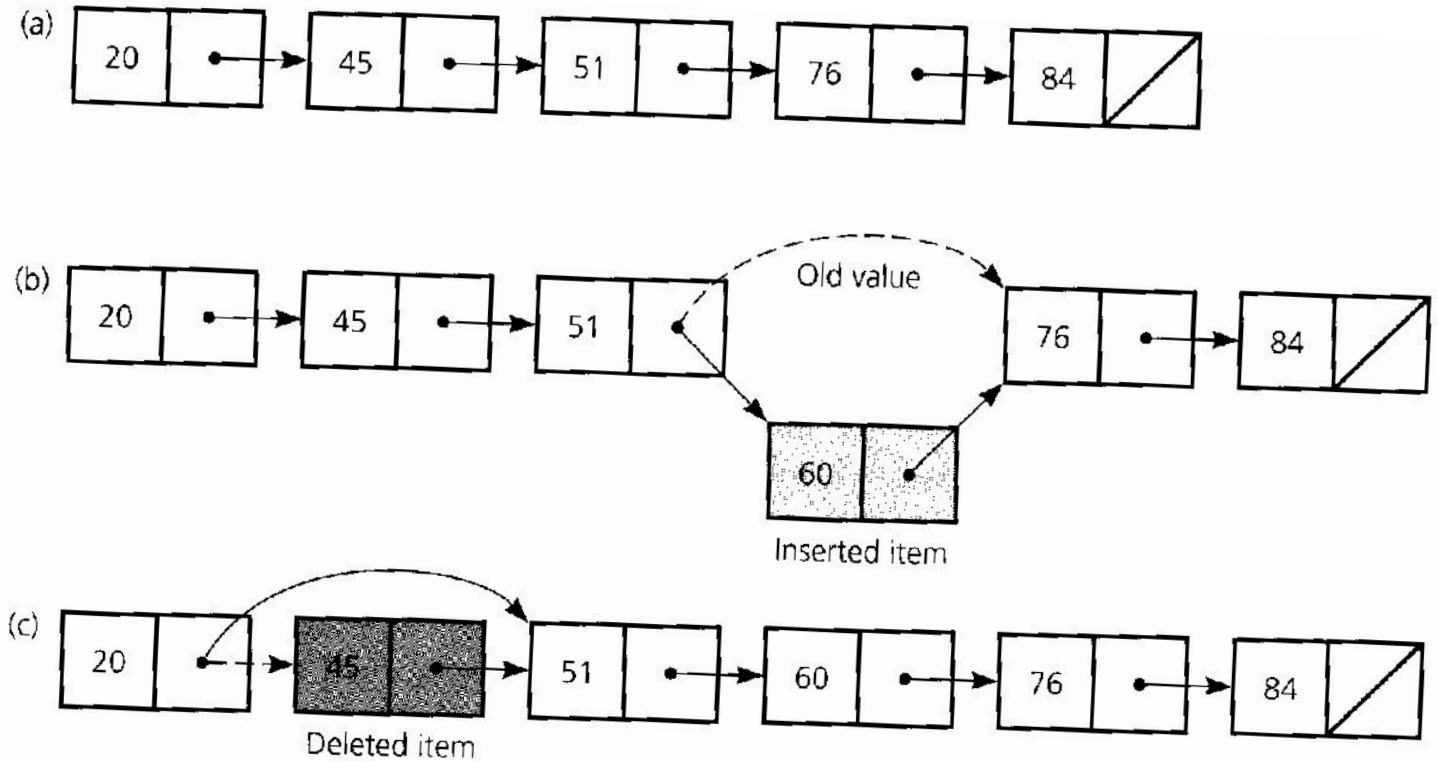
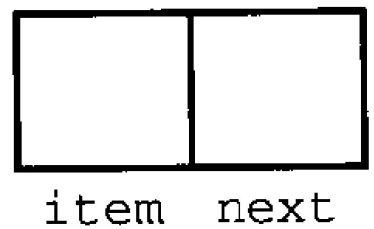


Figure 4-1

(a) A linked list of integers; (b) insertion; (c) deletion

ADT Linked List (Basics)

```
struct Node
{   int    item;
    Node *next;
};   // end struct
```




```
Node *p;
```

Defining a pointer to a node

```
p = new Node;
```

Dynamically allocating a node

```
p->item
```

Referencing a node member

ADT Linked List (Basics)

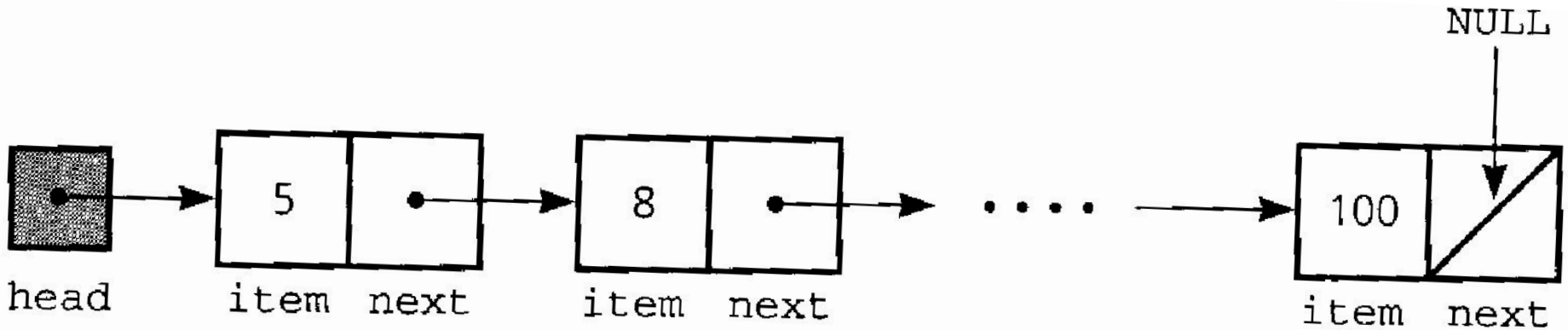
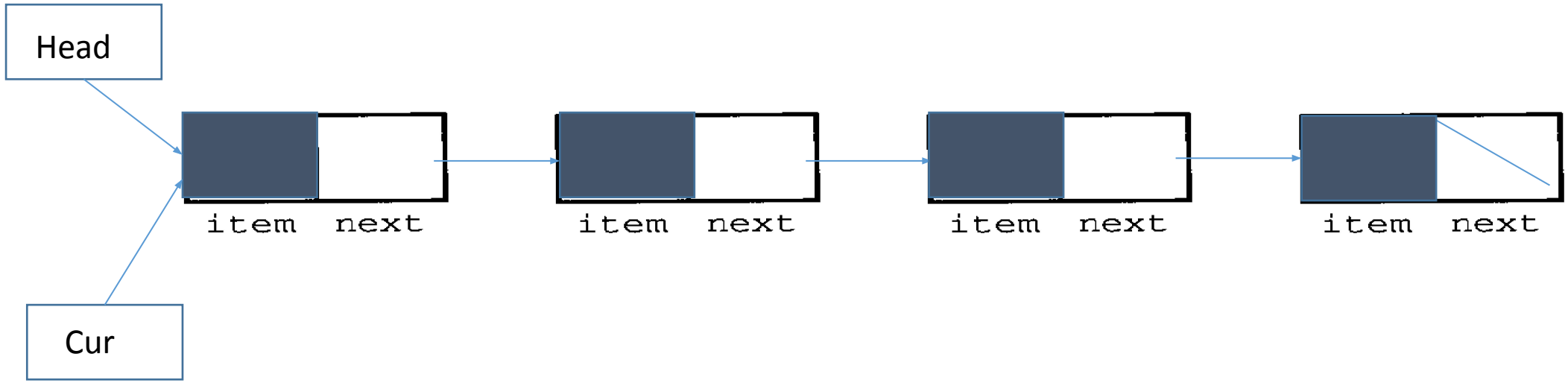


Figure 4-7

A head pointer to a list

Linked List




```
Node *cur = head;
```

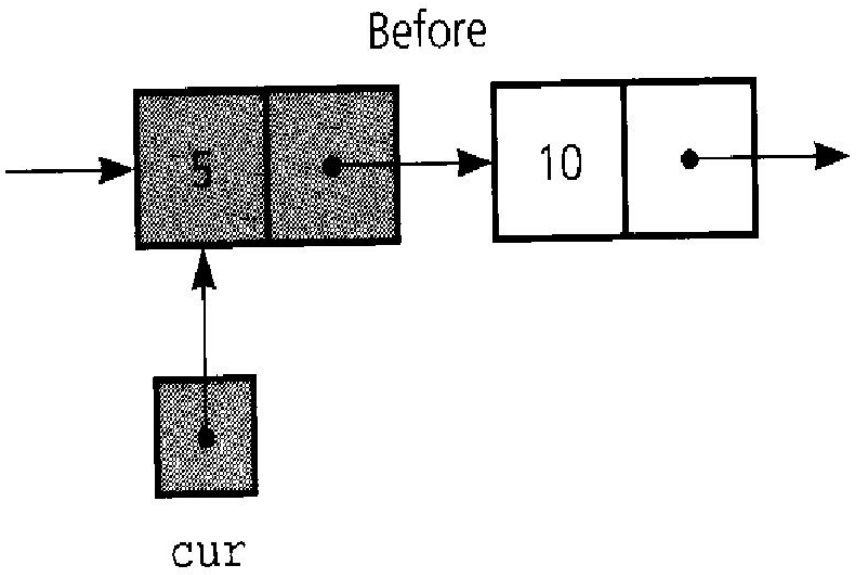
- To Keep track of the current position in the link list, you need a pointer called “cur” that points to the node under consideration.

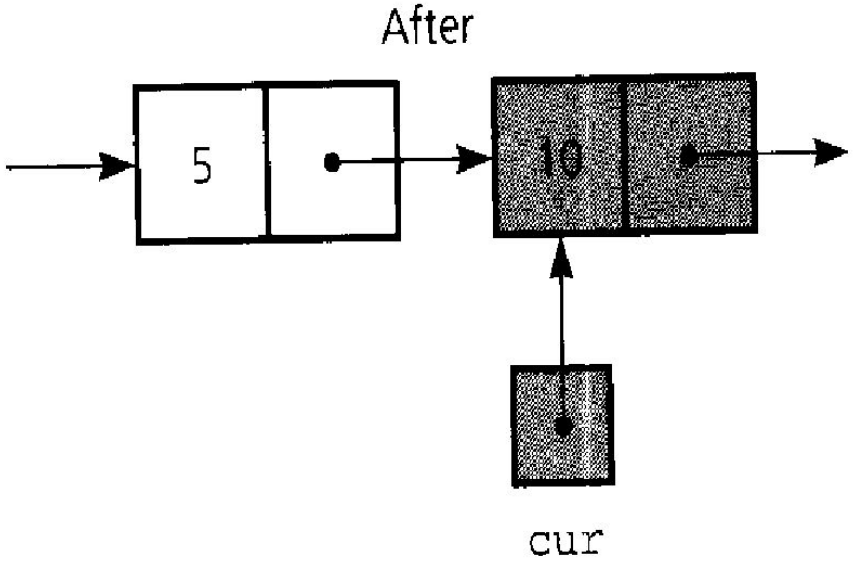
Linked List Basic Operations

- Linked list traversal
- Inserting first node
- Inserting Nth node
- Inserting last node
- Deleting first node
- Deleting Nth node
- Deleting last node

Linked List traversal

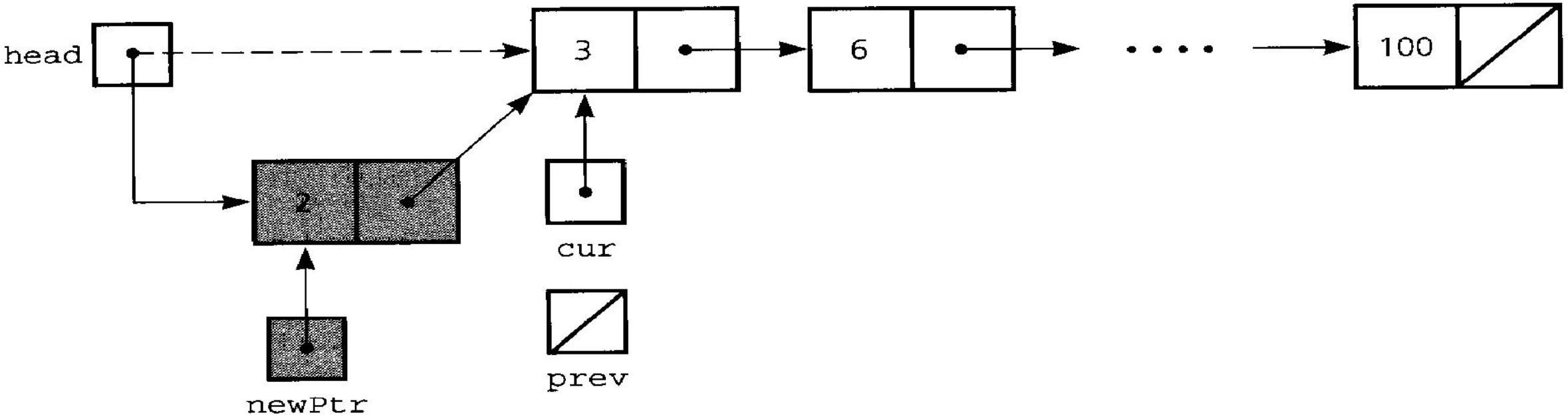
- Linked list traverse operation





cur = cur -> next

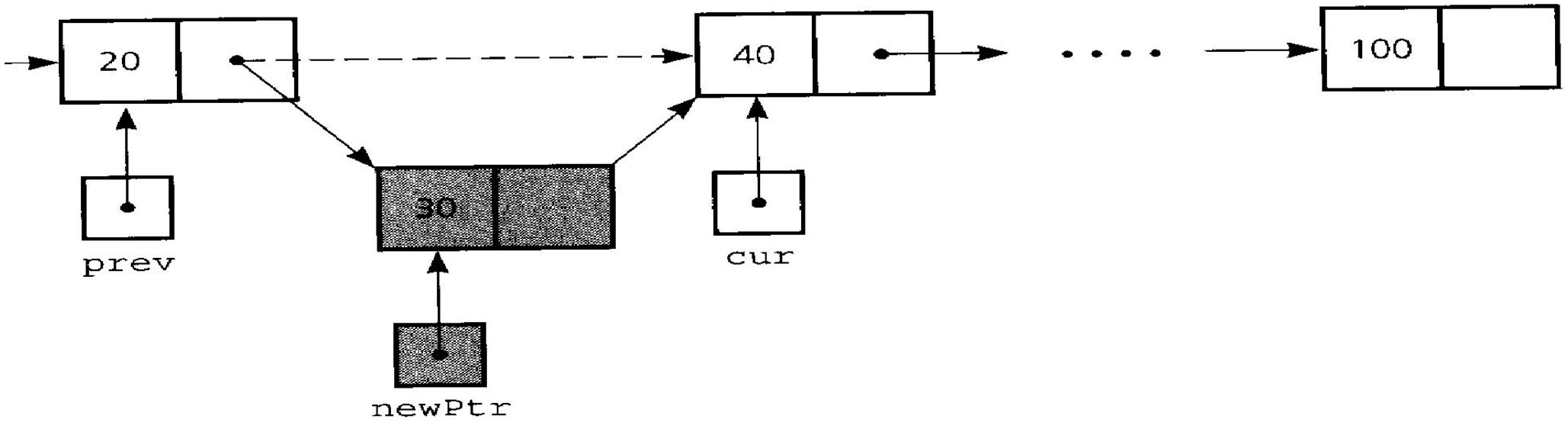
Linked List (Inserting first node)



newPtr= new Node; // create a new “Node” to which “newPtr” points
(this is not the creation of a new pointer)

newPtr -> next = head;
head = newPtr;

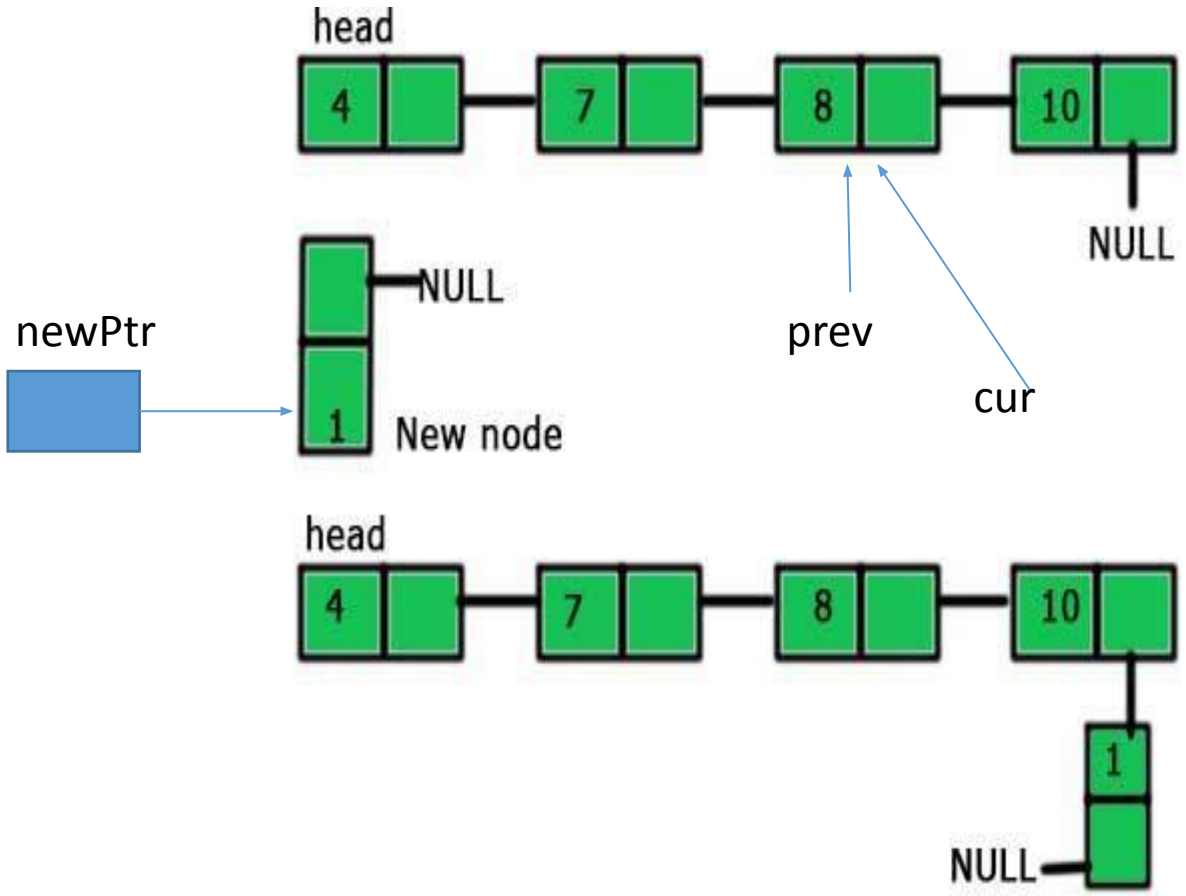
Linked List (Inserting Nth node)



newptr-> next = cur;

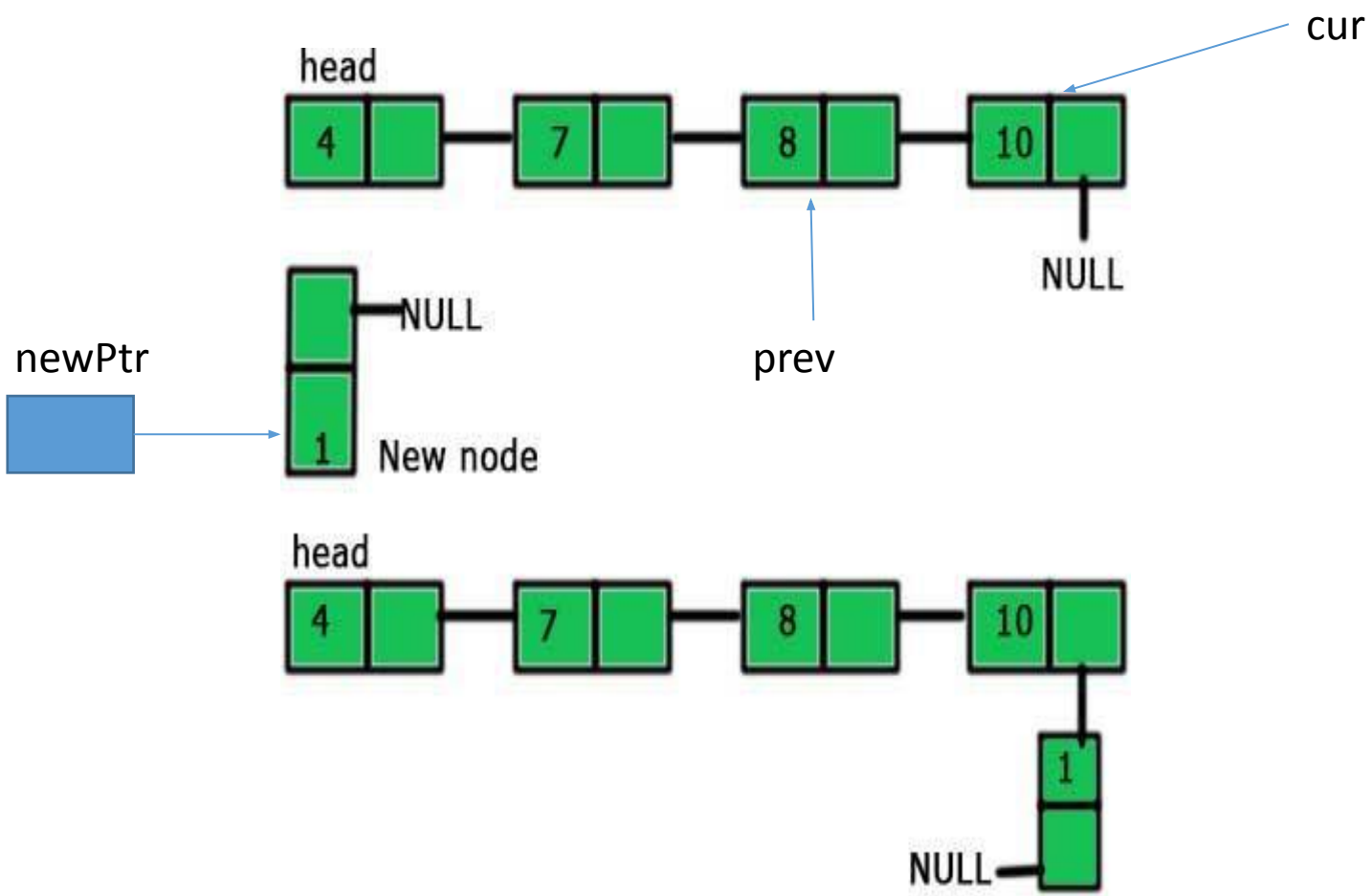
prev-> next = newptr;

Linked List (Inserting Last node)




```
node *newPtr = new node;

// Traverse and find the second last node
while (cur -> next->next != NULL)
        prev= cur;
        cur = cur->next;

//prev->next->next= newPtr;
```

Linked List (Inserting Last node)



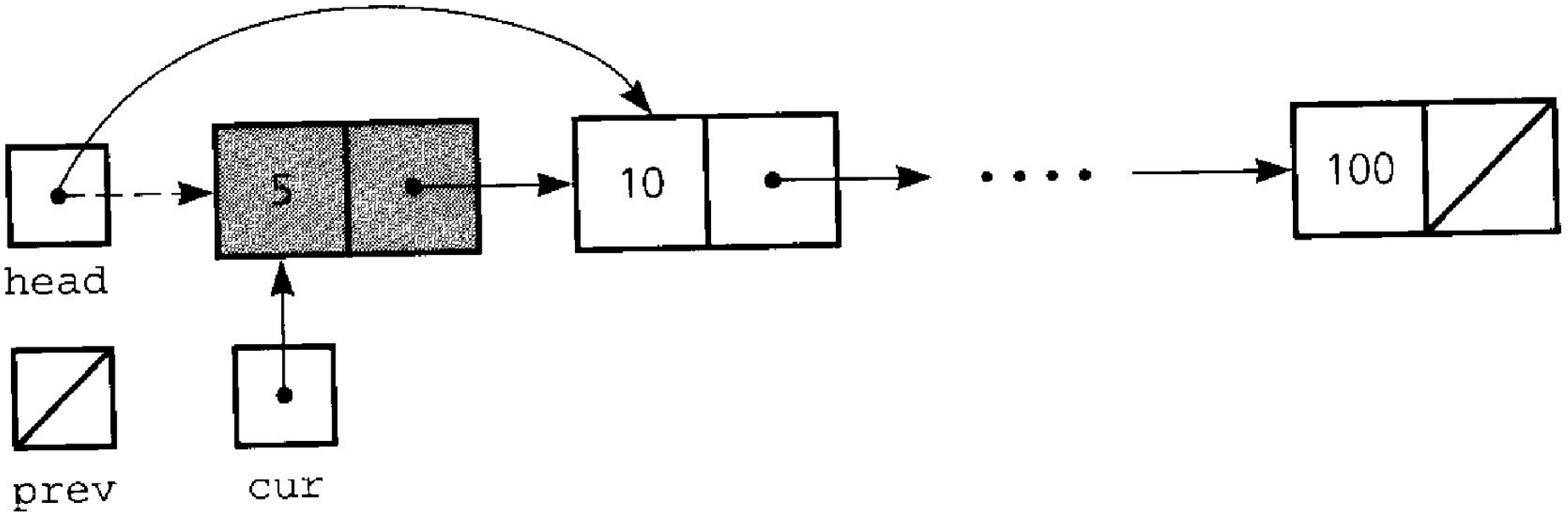

```
//Node *newPtr = new node;

// Cur pointer points to the last node
//while (cur -> next->next != NULL)
        //prev= cur;
        //cur = cur->next;

prev->next->next= newPtr;
```

Or

```
cur->next= newPtr;
```

Move cur and prev pointer to the successor nodes.

Linked List Deleting first node

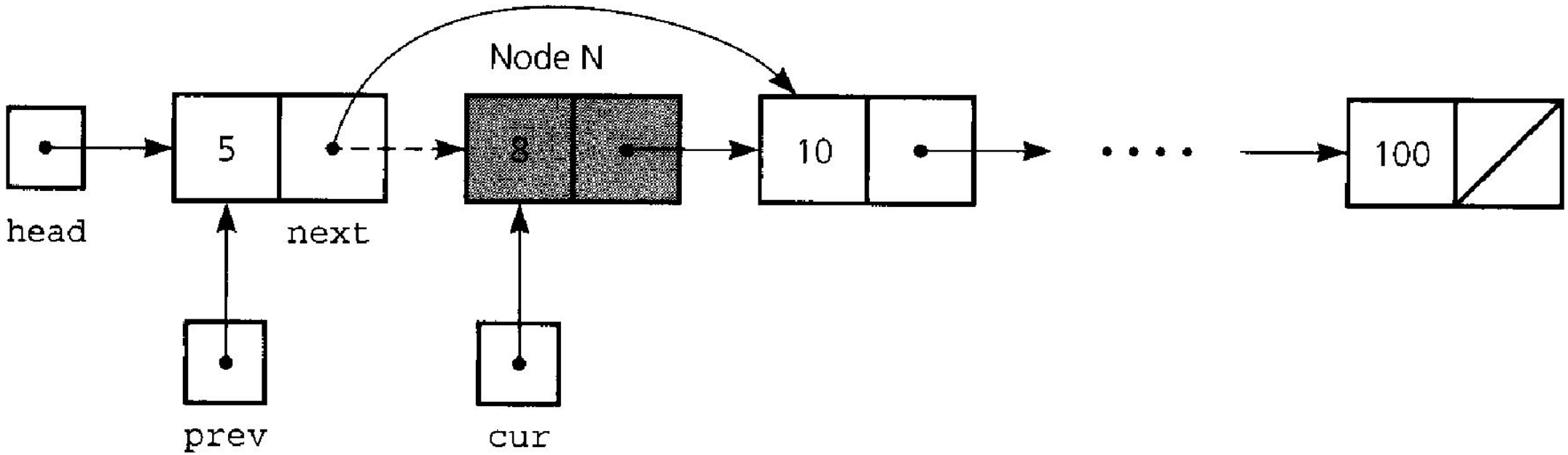


head = head->next

cur -> next = NULL
del cur
cur = NULL

//can make a temp pointer instead of using cur and free(temp).

Linked List Deleting Nth node



(2 pointers are required for Nth node deletion, cur and prev)

prev -> next = cur -> next

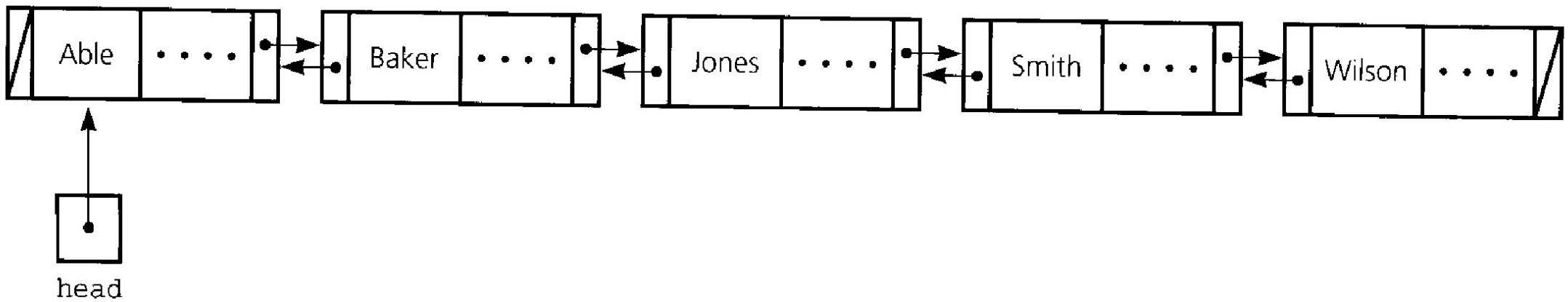
Linked List Deleting Last node

```
//Node* cur = head;

// Traverse the list and find the second last node
while (cur ->next->next != NULL)
    cur = cur->next;

delete (cur ->next);            // Delete last node
cur ->next = NULL;              // Change next of second last node
```

Linked List Deletion of Node (Summary)

- Locate the node you want to delete.
- Disconnect the node from list by changing pointers.
- Return the node to the system.

Doubly Linked List



Doubly Linked List (Applications)

- Doubly linked list can be used in navigation systems where both front and back navigation is required.
- It is used by browsers to implement backward and forward navigation of visited web pages i.e. back and forward button.
- It is also used by various application to implement Undo and Redo functionality.